护栏：错误、重试与审批门控

讲座 11 · 2026 年 10 月 6 日

layout: default

一轮，三个检查点

护栏放在哪里



护栏是你代码中的一个检查。模型无法用言语绕过它。

layout: section

第一部分：当某事失败时

错误与重试

layout: default

在处理之前先分类

三种失败



layout: default

10 月 6 日实测 · 一个对所有请求返回 502 的模型端点

一个死端点，30 个请求



Codex 0.160.0 · **6 次尝试 × 5 次发送 = 30** · 然后退出状态 1

layout: default

现场 · 两个终端

观看 Codex 放弃

左侧：一个总是失败的模型

```
$ python3 fake-endpoint.py 8799 502
```

- 五次爆发，间隔增长

右侧：真实的 Codex，指向它

```
$ ./codex-502.sh 8799
```

- `Reconnecting... 1/5` 到 `5/5`
- 最后一行：`codex exit status: 1`

layout: default

为你的智能体

你拥有的重试循环

```
RETRY = {429, 500, 502, 503, 504}

def call_model(send, request, deadline_s=60):
    start, delay = time.monotonic(), 0.5
    while True:
        try:
            return send(request)                    # 使用 max_retries=0 构建的客户端
        except ModelHTTPError as e:
            if e.status not in RETRY:
                raise                               # 400、401、太长：停止
            wait = e.retry_after or delay * random.uniform(0.5, 1.5)
            if time.monotonic() - start + wait > deadline_s:
                raise                               # 放弃，并以非零状态退出
            time.sleep(wait)
            delay = min(delay * 2, 16)
```

layout: default

工具侧

超时不是无操作

```
step 6   run_command   taskr add "buy milk"
         killed by the harness after 30 s
```

- 它保存了吗？框架无法知道
- 重试它，可能有两个
- 返回："30 秒后超时。它可能已运行。"

只有在运行两次等同于运行一次时才重试调用。

layout: section

第二部分：你审批什么

当智能体询问你时

layout: two-col

一个编辑

```
write_file(path, content)
```

变更在调用中。你读取一个 diff，git 可以将其恢复。

::right::

一个命令

```
run_command(command)
```

调用是一个程序。你看到运行什么，看不到它 touches 什么。

相同的 y/n 提示，两种不同的工作。

layout: default

命令提示隐藏的三件事

提示不显示什么

	你看到	你看不到
	"清理测试状态"	它是由编写命令的模型编写的
	`python -c "...main(['add', ...])"`	它保存到用户的 `~/.taskr.json`
	`pytest`，在第 12 步	`tests/conftest.py`，在第 11 步写入

审批运行中的命令状态。永远不要批准解释。

layout: default

Codex 0.160.0 · Aider 5dc9490b

Codex 和 Aider 询问方式不同

Codex：一个沙箱，然后一个提示

- `--sandbox`：`read-only`，`workspace-write`，`danger-full-access`
- `--ask-for-approval`：`on-request` 或 `never`
- 只有离开沙箱的内容才询问

Aider：只有一个提示

- `(Y)es/(N)o/(D)on't ask again [Yes]`
- 回车表示是
- `--yes-always`：shell 命令得到 否

layout: default

中场休息

你没有阅读就批准的内容

挑选一件 Codex 或 Claude Code 这周要求你批准而你未经阅读就批准的事情。

一个程序需要知道什么才能为你回答它？

layout: section

第三部分：审批门控

当其他东西回答时

layout: default

为什么需要审批门控

大多数提示得到 yes

93%
的 Claude Code 提示被批准
720
10,000 次 Codex 操作中离开沙箱的次数
7
这 720 次中被审查器拒绝的次数
一个几乎总是 yes 的提示停止被阅读。

layout: default

三样东西可以回答

三种审批门控

	代码规则	审查模型	类型化分类器
	判断	文本	意图和效果	意图和效果
	返回	匹配或不匹配	一个决策，一个原因	概率
	每次调用	免费	一次模型调用	70 到 500 毫秒，供应商
	示例	前缀规则	Codex 自动审查	Jev

layout: default

两个已发布的审查模型

自动模式和自动审查

		Claude Code 自动模式	Codex 自动审查
	到达它的内容	通过允许列表和仓库内编辑	只有离开沙箱的内容
	审查器	Sonnet 4.6，两个阶段	GPT-5.4 Thinking，低精力
	一个拒绝	一个工具结果，带原因	一个工具结果，带原因
	停止运行	连续 3 次拒绝	连续 3 次拒绝

layout: default

Claude Code 自动模式

审查器看到的内容



layout: default

TypeSafe · 2026 年 9 月 15 日

Jev，一个类型化决策模型

- 输入：一个状态，作为文本或 JSON
- 输出：带概率的类型化答案
- 没有文本，所以没有什么可解析的
- 供应商：每次请求 70 到 500 毫秒

早期访问。这些数字是 TypeSafe 的，不是我们的。

layout: default

Choice ‘Noul ‘Score

安全类别，作为问题

```
QUESTIONS = {
    "kind": Choice(
        instructions="What does `call` do when it runs?",
        criteria={"read": "Only reads files in the repo",
                  "edit": "Changes files in the repo",
                  "run": "Runs code, tests or a build",
                  "outside": "Touches anything outside the repo"}),
    "asked": Noul(instructions="Did a user message ask for this action?"),
    "undo": Score(
        instructions="If `call` is a mistake, how hard is it to undo?",
        criteria=["Nothing changes", "git undoes it",
                  "Fixable by hand", "Cannot be undone"]),
}
```

layout: default

代码优先，然后 Jev，然后人

代码和 Jev 中的审批门控



layout: default

同一个门控，用 Python

阈值是代码

```
def gate(call, user_msgs):
    if outside_repo(call) or denied_by_rule(call):
        return "deny"                          # 代码：没有询问任何人
    a = jev.system_one(state={"user": user_msgs, "call": call},
                       questions=QUESTIONS).answers
    undo, asked, kind = a["undo"].score, a["asked"].noul, a["kind"]
    if undo >= 2 and asked < 0.5:
        return "deny"                          # 难以撤销，不是被要求的
    if kind.choice == "read" and kind.confidence > 0.9:
        return "allow"
    if undo < 1 and asked > 0.9:
        return "allow"
    return "ask"                               # 中间带
```

layout: default

在哪里画线

一个分数，两个策略

两条线将一个概率转化为三个答案：运行它、询问一个人、阻止它。
TypeSafe 食谱，严格
blocked
block at 0.70
0.74
被阻止
TypeSafe 食谱，宽松
blocked
block at 0.85
0.74

layout: default

审批门控不能承诺什么

测量，不是信任

	Claude Code 自动模式	安全操作被阻止	过度热心放行
	两个阶段	0.4%	17%
	仅第一阶段	8.5%	6.6%

	Codex 自动审查	拒绝
	合成过度热心操作	90.3%
	合成提示词注入	99.3%

审批门控是一个分类器。分类器被评估。

layout: default

这周

这周的日期

	时间	内容
	今晚，11:59 PM	Apply 构建到期
	周四	Analyze 规范，在 L12 之后
	你的实验课04 之前	技能设计文档起草完成
	10 月 12 日或 13 日	实验课04

layout: default

带走这些问题

三个问题

1. 你的智能体重试什么，重试多久？
2. 哪些审批可以用代码回答？
3. 谁回答中间带？

layout: end